UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 3

Student Name: Nitin Sharma **UID: 25MCA20226**
Branch: Computer Application **Section/Group: 5-B**
Semester: 2nd **Date of Performance:**
Subject Name: Automation Software Testing **Subject Code:25CAP-702**

Aim/Overview of the practical:

- To perform SOAP API testing using SoapUI and validate web services by verifying request-response structures, WSDL operations, and functional correctness.
- To perform REST API testing using Postman by executing GET, POST, PUT, and DELETE requests and validating responses, status codes, headers, and payload data.

Requirements:

- SoapUI installed`
- Internet connection
- Sample SOAP Web Service WSDL

Example WSDL:

http://www.dneonline.com/calculator.asmx?WSDL

Procedure:

- Open **SoapUI**.
- Select **File → New SOAP Project**.
- Enter a project name, for example, calculator
- Paste the WSDL URL.
- Click **OK**.

Implementation:

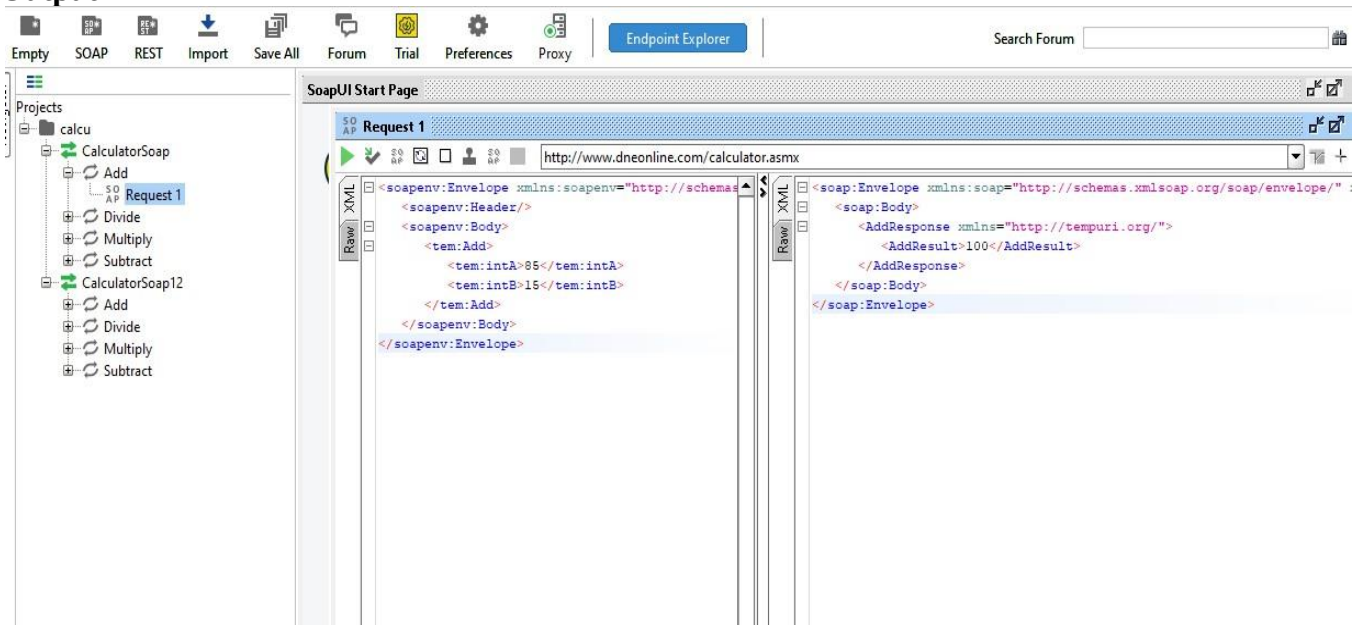
1. Soap Request (ADD)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tem="http://tempuri.org/">
  <soapenv:Header/>
  <soapenv:Body>
    <tem:Add>
      <tem:intA>85</tem:intA>
      <tem:intB>15</tem:intB>
    </tem:Add>
  </soapenv:Body>
</soapenv:Envelope>
```

Soap Response

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <AddResponse xmlns="http://tempuri.org/">
      <AddResult>100</AddResult>
    </AddResponse>
  </soap:Body>
</soap:Envelope>
```

Output



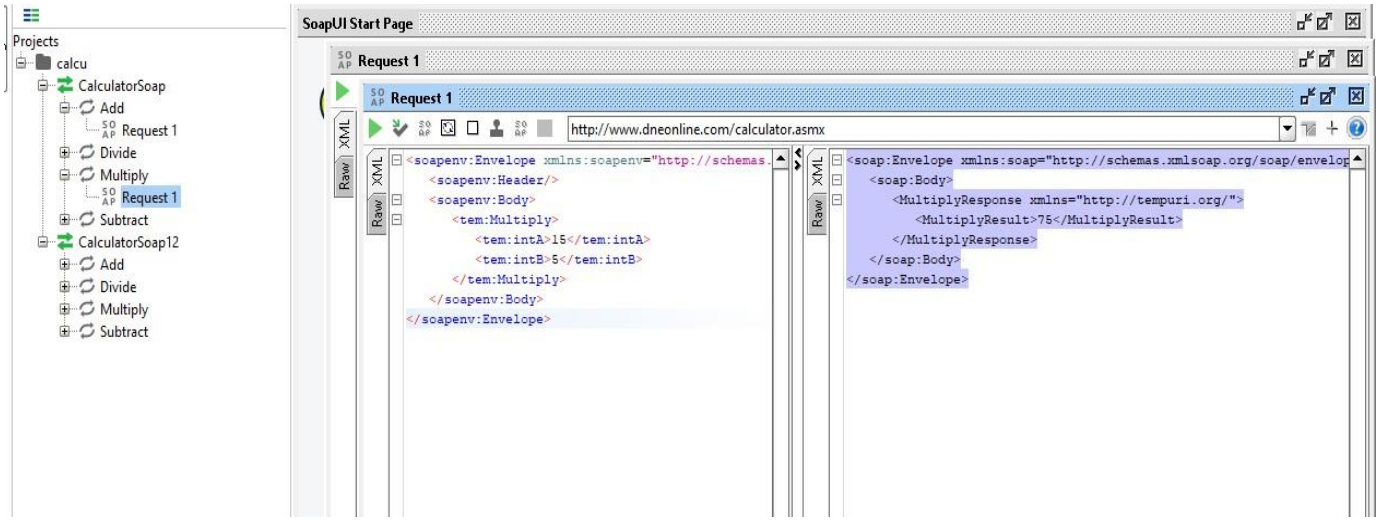
2 .Soap Request (Multiply)

```
<soapenv:Envelope  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"  xmlns:tem="http://tempuri.org/">
  <soapenv:Header/>
  <soapenv:Body>
    <tem:Multiply>
      <tem:intA>15</tem:intA>
      <tem:intB>5</tem:intB>
    </tem:Multiply>
  </soapenv:Body>
</soapenv:Envelope>
```

Soap Response

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <MultiplyResponse xmlns="http://tempuri.org/">
      <MultiplyResult>75</MultiplyResult>
    </MultiplyResponse>
  </soap:Body>
</soap:Envelope>
```

Output



Validation Checklist:

Test	What to verify	Expected Result
WSDL Validation	Service Load Correctly	WSDL is imported Successfully
Operation validation	Add exists	Operation is available
Operation validation	Multiply exists	Operation is available
Request validation	SOAP envelope and body	Correct XML structure
Functional validation	Input 85 + 15	Result = 100
Functional validation	Input 15 × 5	Result = 75

Conclusion

The SOAP web service was successfully tested using SoapUI. The WSDL operations were verified, SOAP request and response structures were checked, and functional correctness was validated using number conversion test cases**.**

(B) REST API Testing using Postman

Aim

To perform REST API testing using Postman by executing GET, POST, PUT, and DELETE requests and validating responses, status codes, headers, and payload data.

Requirements

- Postman installed
- Sample REST API

Example API:

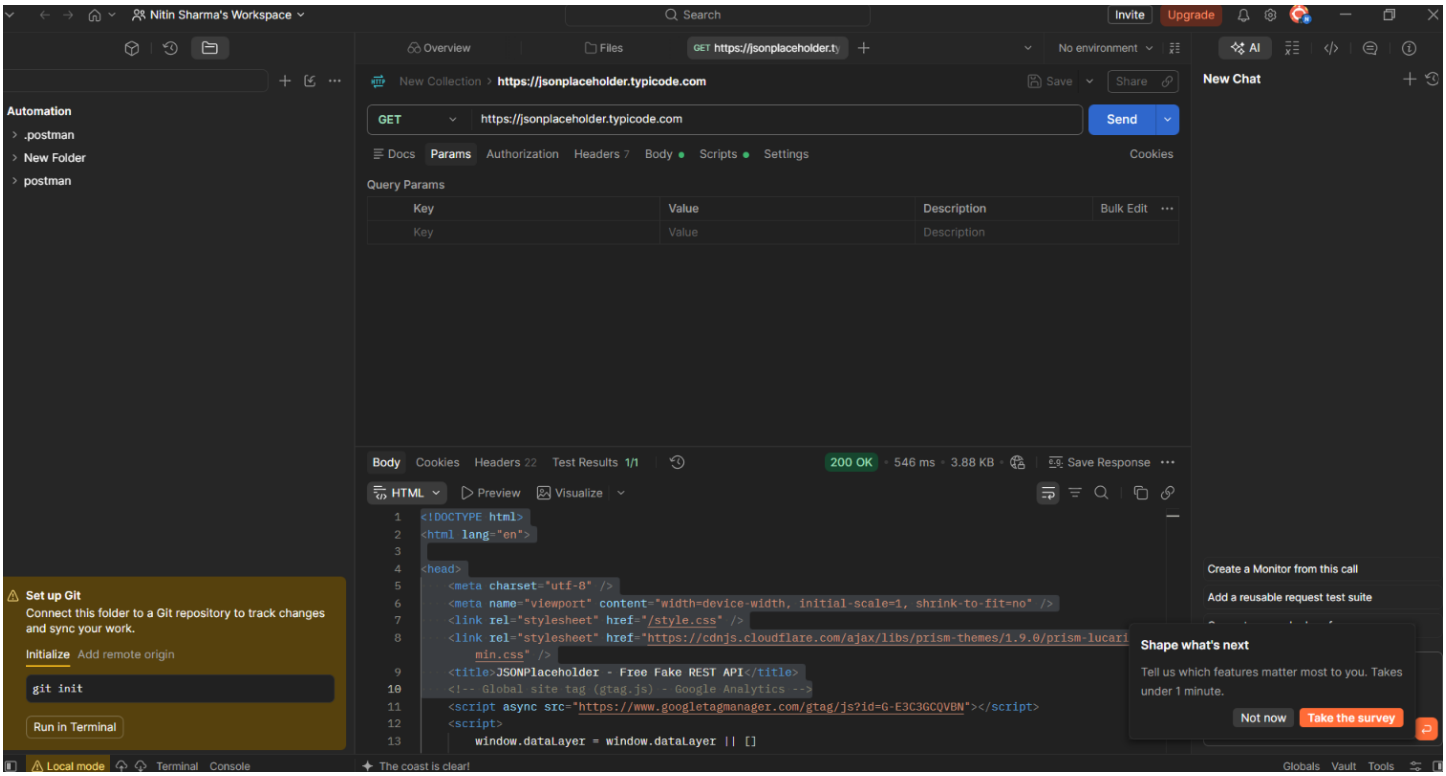
1. GET Request — Retrieve Users

In Postman

1. Open **Postman**.
2. Click **New → HTTP Request**.
3. Select **GET**.
4. Enter:https://jsonplaceholder.typicode.com
5. Click **Send**.

Response

Output



2. POST Request — Create a User
In Postman

1. Change method to **POST**.
2. Enter: https://jsonplaceholder.typicode.com/posts
3. Click **Body → raw → JSON**.
4. Enter:

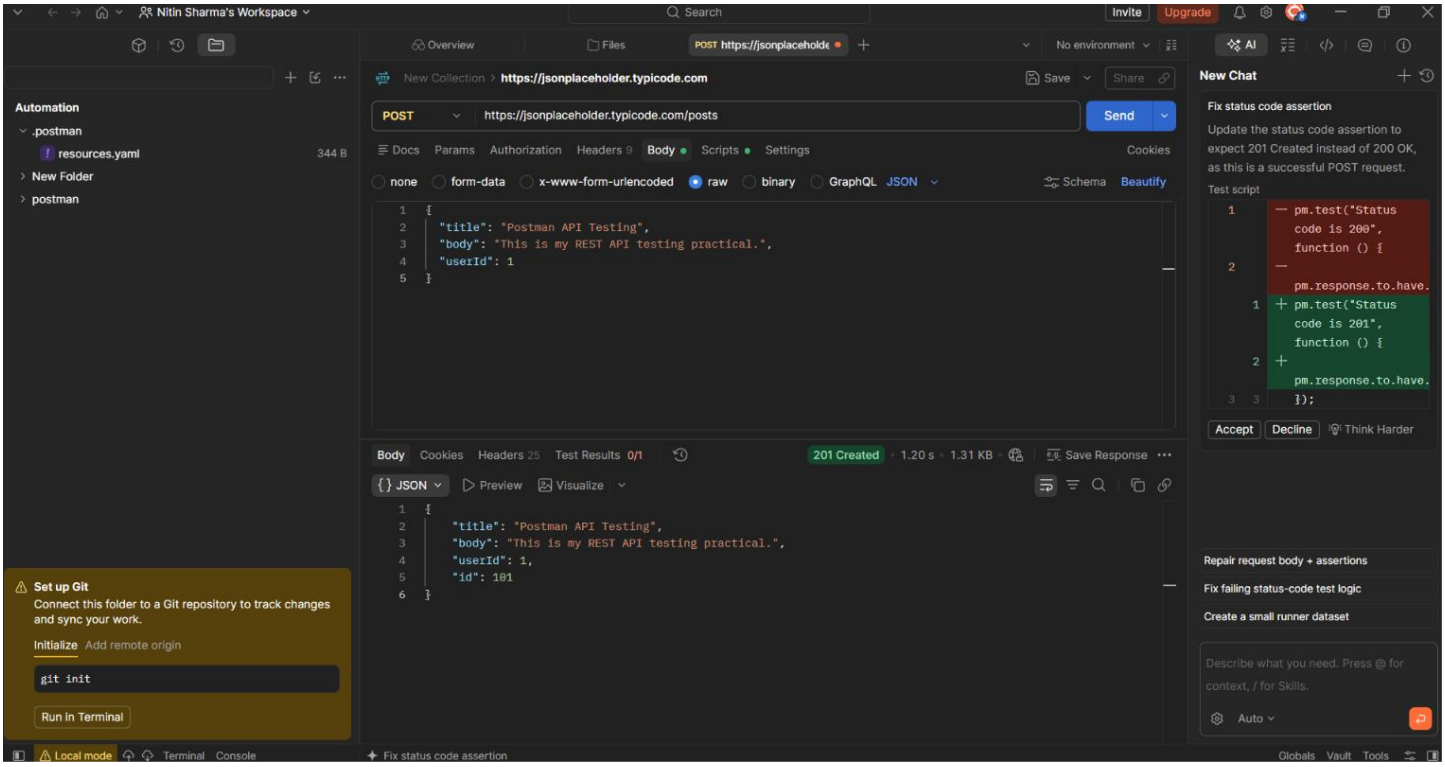
```
{
  "title": "Postman API Testing",
  "body": "This is my REST API testing practical.",
  "userId": 1
}
```

5. Click **Send**.

Response:

```
{
    "title": "Postman API Testing",
    "body": "This is my REST API testing practical.",
    "userId": 1,
    "id": 101
}
```

Output



3. PUT Request — Update a User

In Postman

1. Change method to **PUT**.
2. Enter: https://jsonplaceholder.typicode.com/posts/1
3. Go to **Body → raw → JSON**.

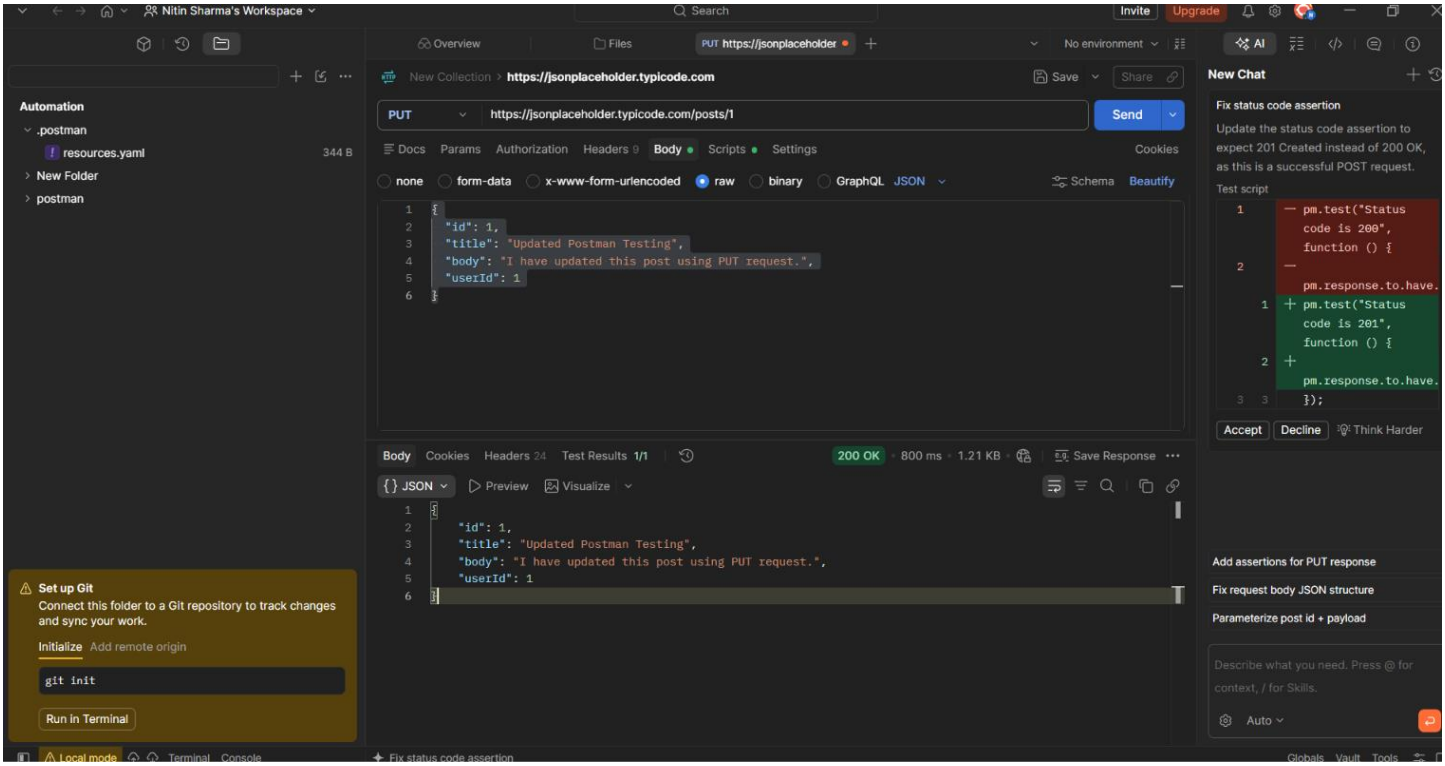
4. Enter:

```
{
  "id": 1,
  "title": "Updated Postman Testing",
  "body": "I have updated this post using PUT request.",
  "userId": 1
}
```

Click **Send**. **Response:**

```
{
    "id": 1,
    "title": "Updated Postman Testing",
    "body": "I have updated this post using PUT request.",
    "userId": 1
}
```

5. **Output**



4. DELETE Request — Delete a User

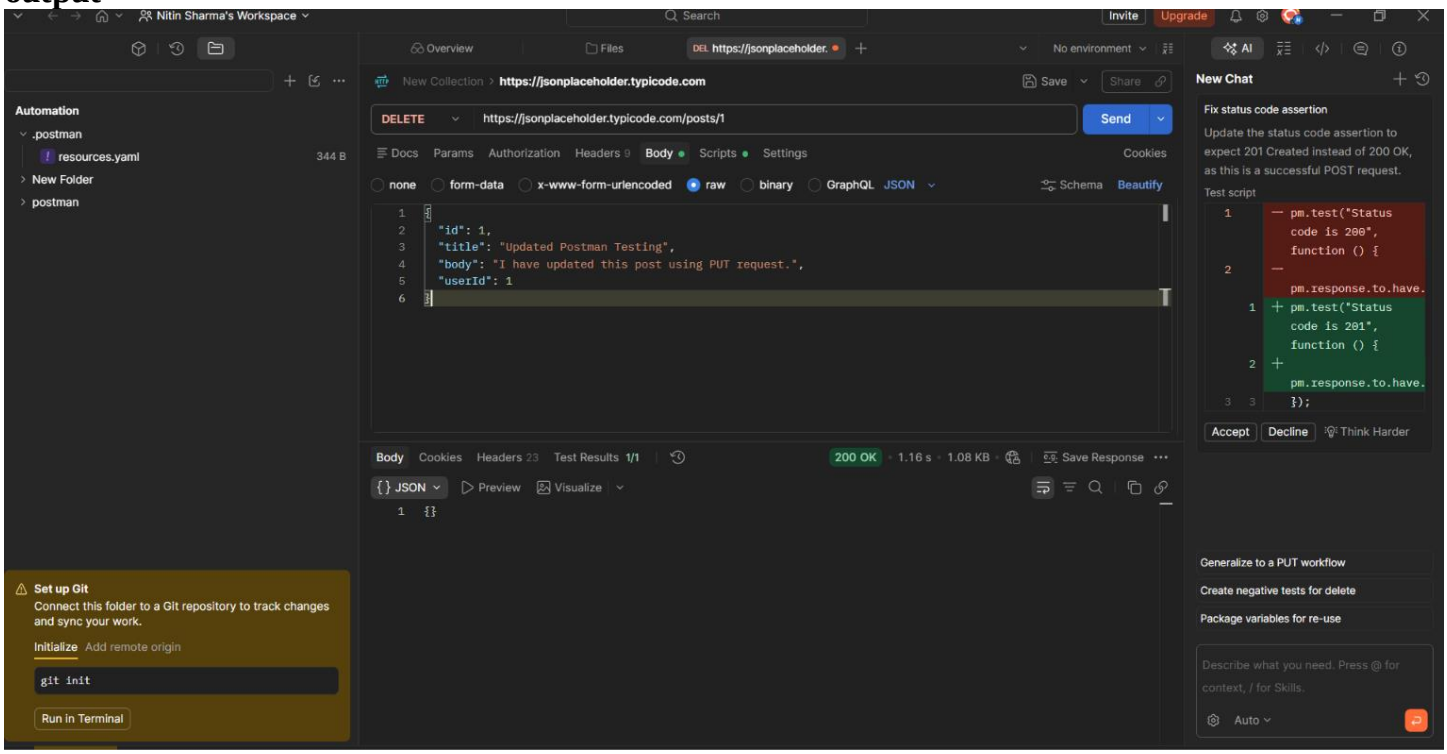
In Postman

1. Change method to **DELETE**.
2. Enter:https://jsonplaceholder.typicode.com/posts/1
3. Click **Send**.

Response:

{}

output



Complete Experiment Table

Test	Method	URL	Expected
Retrieve post	GET	/posts/1	200 OK
Create post	POST	/posts	201 Created
Update post	PUT	/posts/1	200 OK
Delete post	DELETE	/posts/1	200 OK

Conclusion

REST API testing was successfully performed using Postman and the ReqRes API. GET, POST, PUT, and DELETE requests were executed, and responses were validated by checking status codes, headers, JSON payloads, and functional correctness**.**

Learning Outcomes

After completing this experiment, the student will be able to:

1. Understand the basic concept of REST APIs.

2. Perform GET, POST, PUT and DELETE requests using Postman.
3. Test API endpoints of the BugFlow AI application.
4. Understand how data is created, retrieved, updated and deleted through APIs.
5. Check API responses using HTTP status codes and JSON data.
6. Identify errors while testing APIs and verify whether the API is working correctly.